

# Meeting Room Scheduler

## Problem Statement

You are given  $N$  meetings, each with a unique ID, a start time, and an end time. Your task is to write a program that assigns meetings to rooms using a greedy strategy and then answers a series of queries about the resulting schedule.

**Time Limit:** 2 seconds. Solutions that produce correct output but exceed the time limit on large inputs will receive **partial marks** only.

## Meeting Structure

Each meeting should be represented using the following **struct**:

```
struct Meeting {
    int id;           // unique meeting identifier
    int start;        // start time of the meeting
    int end;          // end time of the meeting
    int room;         // assigned room number (initially -1)
    int index;        // original input order (0-indexed)
};
```

- **id:** The unique identifier for the meeting, as given in the input.
- **start:** The start time of the meeting.
- **end:** The end time of the meeting.
- **room:** The room number assigned to this meeting. Initialize to  $-1$  before assignment.
- **index:** The position of this meeting in the original input (0-indexed). Used for tie-breaking when two meetings have the same start time.

## Room Assignment Rules

Rooms are numbered starting from 1. Meetings are first **sorted by their start times**. If two meetings have the **same start time**, they are processed in the order they appear in the input (i.e., the meeting with the smaller **index** comes first).

For each meeting in this sorted order:

- If there exists a room whose previous meeting has already ended (i.e., its end time  $\leq$  current meeting's start time), assign the current meeting to the room that became free **earliest**. If multiple rooms freed at the same time, assign the one with the **smaller room number**.
- If no such room is available, open a new room (incrementing the room counter) and assign the meeting there.

After assignment, update the meeting's **room** field accordingly.

## Input Format

- The first line contains an integer **N** — the number of meetings.
- The next **N** lines each contain three integers: **id**, **start**, **end** — representing the meeting's unique ID, start time, and end time respectively.
- The next line contains an integer **Q** — the number of queries.
- The next **Q** lines each contain one of the commands described below.

## Commands

### `sortedlist`

Print all meetings sorted by **start time** (ascending). If two meetings have the same start time, print them in **input order** (i.e., the meeting with the smaller **index** comes first). Use a custom function object (functor) with `std::sort` to sort the meetings in the expected order before printing. Each line should follow the format:  
Meeting <id> [<start>,<end>]

### `schedule`

Print the full schedule sorted **primarily by room number** (ascending), and **secondarily by start time** (ascending) within the same room. Each line should follow the format:

Meeting <id> -> Room <room> [<start>,<end>]

### `rooms`

Print the total number of rooms used.

### `mostused`

Print the room number that hosts the most meetings. If there is a tie, print the **smallest room number** among them.

### `longest`

Print the ID of the meeting with the longest duration (**end** — **start**). If there is a tie, print the **smallest meeting ID** among them.

find <time>

Print the IDs of all meetings that are **active** at the given time point. A meeting is active at time  $t$  if and only if:

$$\text{start} \leq t < \text{end}$$

A meeting that **ends** at time  $t$  is **not** considered active at  $t$ . Print the IDs **space-separated** on a single line, in **start-time sorted order** (using the same ordering as `sortedlist`). If no meeting is active at the given time, print `None`.

## Output Format

For each query, print the result on a new line as described above.

## Example

### Input

```
6
1 0 10
2 5 15
3 10 20
4 0 5
5 15 25
6 20 30
6
sortedlist
schedule
rooms
mostused
longest
find 5
```

### Output

```
Meeting 1 [0,10]
Meeting 4 [0,5]
Meeting 2 [5,15]
Meeting 3 [10,20]
Meeting 5 [15,25]
Meeting 6 [20,30]
Meeting 1 -> Room 1 [0,10]
Meeting 3 -> Room 1 [10,20]
Meeting 6 -> Room 1 [20,30]
Meeting 4 -> Room 2 [0,5]
Meeting 2 -> Room 2 [5,15]
Meeting 5 -> Room 2 [15,25]
2
```

```
1
1
1 2
```

## Explanation

### Sorting

Meetings sorted by start time: 1(0–10), 4(0–5), 2(5–15), 3(10–20), 5(15–25), 6(20–30). Meetings 1 and 4 both start at 0 — since Meeting 1 has `index = 0` and Meeting 4 has `index = 3`, Meeting 1 is processed first.

### Room Assignment

- **Meeting 1** (0–10): No rooms exist → **Room 1**.
- **Meeting 4** (0–5): Room 1 ends at  $10 > 0$ , not free → **Room 2**.
- **Meeting 2** (5–15): Room 2 ends at  $5 \leq 5$ , free → reuse **Room 2**.
- **Meeting 3** (10–20): Room 1 ends at  $10 \leq 10$ , free → reuse **Room 1**.
- **Meeting 5** (15–25): Room 2 ends at  $15 \leq 15$ , free → reuse **Room 2**.
- **Meeting 6** (20–30): Room 1 ends at  $20 \leq 20$ , free → reuse **Room 1**.

### Query Results

- **sortedlist**: All meetings printed in start-time order. Meetings 1 and 4 both start at 0; Meeting 1 comes first because it has a smaller `index` (0 vs 3).
- **schedule**: Room 1 holds meetings 1, 3, 6. Room 2 holds meetings 4, 2, 5 — each group sorted by start time within the room.
- **rooms**: 2 rooms used.
- **mostused**: Both rooms have 3 meetings — tie broken by smallest room number → **1**.
- **longest**: All meetings have duration 10 — tie broken by smallest meeting ID → **1**.
- **find 5**: Check each meeting for  $\text{start} \leq 5 < \text{end}$ :
  - Meeting 1 (0–10):  $0 \leq 5 < 10$  → **active**.
  - Meeting 4 (0–5):  $0 \leq 5$  but  $5 < 5$  is false → **not active** (ends exactly at 5).
  - Meeting 2 (5–15):  $5 \leq 5 < 15$  → **active**.All others start after 5. Output in start-time sorted order → **1 2**.

## Constraints

- **Time Limit:** 2 seconds
- $1 \leq N \leq 10^5$
- $1 \leq Q \leq 10$
- $0 \leq \text{start} < \text{end} \leq 10^9$
- All meeting IDs are unique positive integers.
- For `find` queries, the given time value satisfies  $0 \leq \text{time} \leq 10^9$ .

## Hint

Consider using `std::set` from the C++ STL to efficiently track active rooms during room assignment.

## Partial Marks Breakdown

Each command is evaluated independently. Correctly implementing a command earns its share of marks, regardless of whether other commands are implemented.

| Component               |
|-------------------------|
| <code>sortedlist</code> |
| <code>schedule</code>   |
| <code>rooms</code>      |
| <code>mostused</code>   |
| <code>longest</code>    |
| <code>find</code>       |

- `sortedlist`, `longest`, and `find` depend only on the meetings array and can be implemented independently of room assignment.
- `schedule`, `rooms`, and `mostused` require correct room assignment to earn full marks.
- Room assignment must be implemented efficiently using optimal techniques to pass all test cases within the time limit.